

UNIVERSIDAD ESTATAL DE MILAGRO

Facultad de Ciencias e Ingeniería (FACI)

Carrera:

Ingeniería en Software

TALLER DE PROGRAMACIÓN ORIENTADAS A OBJETOS

CASO 02 – GESTIÓN DE TURNOS

ESTUDIANTE:

SALMA NUÑEZ OLALLA

DOCENTE:

ING. ERNESTO SEGUNDO GUAMAN UZHCA

AÑO:

2026

Introducción

En la vida cotidiana, situaciones tan comunes como acudir a una ventanilla bancaria, una oficina de atención ciudadana o incluso un servicio de mensajería operan bajo un principio muy simple pero efectivo: la primera persona en llegar es la primera en ser atendida. Este mecanismo, conocido formalmente como FIFO (First In, First Out), constituye uno de los modelos de organización de colas más utilizados tanto en sistemas computacionales como en procesos administrativos. (Cormen et al., 2009)

La simulación computacional de dicho comportamiento utilizando el lenguaje de programación Python, aplicando los fundamentos de la programación orientada a objetos, se fundamenta desde un punto de vista que radica en la necesidad de representar fielmente los elementos intervinientes en un sistema real de turnos: las personas, los turnos asignados y el puesto de atención. Cada uno de estos elementos se debe modelar como una clase independiente, buscando así un código modular, legible y fácilmente extensible. (Hill, 2020)

Objetivos

Objetivo General

- Analizar el proceso de simulación de un sistema de atención por turnos basado en el principio FIFO (First In, First Out), mediante la implementación de un programa en Python con un diseño orientado a objetos.

Objetivos Específicos

- Describir la estructura y responsabilidades de cada una de las clases que componen el programa desarrollado.

- Identificar los atributos y métodos que materializan el comportamiento FIFO dentro de la simulación.

Desarrollo

Previo a la codificación, se realizó un análisis preliminar orientado a identificar los objetos del dominio del problema. Este análisis arrojó tres entidades claramente diferenciadas:

Persona: portadora de datos básicos como nombre y documento de identidad.

Turno: un número secuencial asociado a una persona en el momento de su llegada.

CajaAtención: el punto de servicio que administra la cola, asigna turnos y ejecuta la atención.

Con base en este modelo conceptual, se procedió a implementar cada clase de manera independiente. En primer lugar, se diseñó la clase “Persona”, cuya responsabilidad se limitó a almacenar y proveer acceso controlado a los atributos de nombre y documento. Posteriormente, se creó la clase “Turno”, cuya función es actuar como un contenedor que vincula a una persona con un número de turno específico. Aunque esta clase podría haberse omitido almacenando directamente a la persona junto con su número dentro de la cola, su inclusión respondió a un criterio de claridad semántica: cada turno representa una unidad discreta dentro del sistema. (Hill, 2020)

La clase central del programa es “CajaAtencion”. En ella se encuentra una lista (self._turnos) que actúa como estructura de cola, así como un contador (self._contador) encargado de generar números de turno de forma incremental. El método agregar_persona incrementa el contador, crea un nuevo objeto Turno y lo añade al final de la lista, respetando así el orden

de llegada. Por su parte, el método “atender_siguiente” extrae y retorna el primer elemento de la lista mediante pop(), operación que materializa el comportamiento FIFO.

Para efectos de simulación, se implementó una función main() que crea cuatro instancias de “Persona” con datos de ejemplo, las registra en la caja de atención y posteriormente las atiende en grupos de dos, mostrando en cada paso el estado de la cola. Esta dinámica permite observar visualmente cómo evoluciona el sistema a medida que se presta servicio. Cabe señalar que, si bien el uso de “pop” sobre una lista resulta ineficiente para colas de gran tamaño, para los propósitos educativos y el volumen reducido de datos del presente ejercicio resulta perfectamente aceptable. (Hill, 2020)

Explicación del código y las clases

- Clase “Persona”

```
class Persona:
    def __init__(self, nombre, documento):
        self._nombre = nombre
        self._documento = documento

    @property
    def nombre(self):
        return self._nombre

    @property
    def documento(self):
        return self._documento
```

La clase Persona tiene como única responsabilidad representar a un individuo que acude a la caja de atención. Para ello, su método constructor “__init__” recibe dos parámetros: el

nombre y el número de documento de la persona. Estos valores se almacenan internamente en los atributos `_nombre` y `_documento`, respectivamente. El uso del prefijo guion bajo es una convención en Python que indica que dichos atributos deben ser tratados como protegidos, es decir, no se pretende que sean accedidos o modificados directamente desde fuera de la clase. Para permitir su lectura de manera controlada, se emplean los decoradores `@property` sobre los métodos `nombre` y `documento`. De este modo, desde el exterior se puede escribir, por ejemplo, `persona.nombre` para obtener el nombre, pero no se podrá asignar un nuevo valor mediante `persona.nombre = "Otro"` a menos que se defina explícitamente un método `setter`. Esta decisión de diseño obedece al principio de encapsulamiento, que busca proteger los datos internos de un objeto y exponer únicamente aquello que resulta necesario para su interacción con otras partes del programa.

- **Clase “Turno”**

```
from modelo.persona import Persona

class Turno:
    def __init__(self, persona, numero):
        self._persona = persona
        self._numero = numero

    @property
    def persona(self):
        return self._persona

    @property
    def numero(self):
        return self._numero
```

En esta clase se recibe dos argumentos: una instancia de la clase `Persona` y un número entero que representa el turno correlativo. Estos valores se almacenan en los atributos

protegidos `_persona` y `_numero`, respectivamente. Al igual que en la clase anterior, se definen propiedades (`@property`) para ambos atributos, de manera que desde el exterior se pueda acceder a la persona asociada al turno y al número del mismo, pero no se puedan modificar una vez que el turno ha sido creado. En este sentido, la clase `Turno` actúa como una "etiqueta" que ordena la información y facilita la legibilidad del programa.

- **Clase “CajaAtencion”**

```
from modelo.turno import Turno

class CajaAtencion:
    def __init__(self):
        self._turnos = []
        self._contador = 0

    def agregar_persona(self, persona):
        self._contador += 1
        nuevo_turno = Turno(persona, self._contador)
        self._turnos.append(nuevo_turno)
        print(f"Turno #{nuevo_turno.numero} asignado a {persona.nombre}")

    def atender_siguiente(self):
        if self.esta_vacia():
            print("No hay nadie en la cola")
            return None

        turno_atendido = self._turnos.pop(0)
        print(f"Atendiendo a {turno_atendido.persona.nombre} (Turno #{turno_atendido.numero})")
        return turno_atendido

    def esta_vacia(self):
        return len(self._turnos) == 0

    def proximo_turno(self):
        if self.esta_vacia():
            print("Cola vacía")
            return None

        siguiente = self._turnos[0]
        print(f"Próximo turno: #{siguiente.numero} - {siguiente.persona.nombre}")
        return siguiente
```

El primer método “agregar_persona(self, persona)” se implementa con una instancia de Persona como argumento y ocurren tres acciones de manera secuencial: primero, el contador se incrementa en una unidad; segundo, se crea un nuevo objeto de la clase Turno, al cual se le pasa la persona recibida y el nuevo valor del contador; tercero, este objeto “Turno” recién creado se agrega al final de la lista “turnos” mediante el método “append”. Este mecanismo respeta el principio FIFO, puesto que los turnos se van acumulando en el orden exacto en que las personas van llegando. Además, el método imprime en pantalla un mensaje informativo que indica qué número de turno ha sido asignado a qué persona.

El segundo método relevante es “atender_siguiente(self)” que verifica si la cola está vacía mediante una llamada al método auxiliar “esta_vacia(self)” con un retorno “True” si la longitud de es igual a cero y “False” en caso contrario. De este modo, la persona que más tiempo ha estado esperando es la primera en salir de la cola. El método imprime un mensaje indicando a quién se está atendiendo y con qué número de turno, y finalmente retorna el objeto Turno atendido (aunque en esta simulación el valor retornado no se utiliza más allá de mostrarlo, podría ser empleado en versiones futuras para registrar la atención o generar estadísticas).

El tercer método, proximo_turno(self), permite consultar cuál es el siguiente turno sin eliminarlo de la cola. Este método resulta especialmente útil para simular el anuncio de "próximo turno" que suele hacerse en los sistemas reales de atención, donde se llama a la siguiente persona mientras la actual está siendo atendida o antes de que comience la siguiente ronda de atención.

Conclusiones

- El análisis permitió describir que la clase Persona almacena los datos básicos del individuo, la clase Turno actúa como un contenedor que asocia a cada persona con un número correlativo, y la clase CajaAtencion administra la cola y ejecuta la atención. Cada clase posee una responsabilidad única y claramente delimitada, lo que facilita la comprensión global del sistema y su mantenimiento futuro.
- Se identificó que el atributo _turnos (una lista) constituye la estructura de cola, mientras que el método agregar_persona añade nuevos turnos al final mediante append. Por su parte, el método atender_siguiete extrae el primer elemento con pop(0). Estas dos operaciones, en conjunto, materializan de manera rigurosa el comportamiento FIFO, garantizando que la primera persona en llegar sea siempre la primera en ser atendida.
- Si bien la simulación cumple con los objetivos planteados, se sugiere como mejora incorporar la lectura de datos desde archivos CSV, lo que permitiría procesar un número variable de personas sin modificar el código fuente. Esta integración facilitaría la realización de pruebas con escenarios más complejos y volúmenes de datos realistas, mejorando así la robustez y escalabilidad del sistema desarrollado.

Bibliografía

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2009). *Introduction to algorithms* (3.^a ed.). MIT Press.
- Hill, C. (2020). *Learning scientific programming with Python* (2.^a ed.). Cambridge University Press.